

COPILOTTRAINING
TECHNICAL BRIEFING

Custom Agents

One File, Every Runtime

Design the portable core. Read the resolved run. Place each requirement where it lives.

Foundations · CopilotTraining Tech Talk · 45 min

🤔 THE CORE QUESTION

"How does a team design one custom agent that behaves as intended in every runtime it reaches?"

Each runtime resolves the same file in its own context: fields, tools, model, and environment.

Write the portable core, then read the resolved run.

Agent producer

Platform engineer who writes the agent file the whole team shares

Developer starting runs

Calls the agent from VS Code or the CLI and owns the launch controls

Maintainer and team lead

Prepare the cloud environment, accept PRs, and set runtime policy

1 file · 3 runtimes

test-writer.agent.md invoked in VS Code, Copilot CLI, and cloud agent (observed)

6 aliases → 23 · 14 tools

Each runtime's own expansion: VS Code 23, Copilot CLI 14 (observed)

Stopped → 6 passed

Cloud Jest after copilot-setup-steps.yml: 1 suite, 6 tests passed (observed)

TABLE OF CONTENTS

PART 1

Design the Portable Core

Description selects, tools bound, body teaches

Role, tool boundary, method, and output contract in the shared core

PART 2

Run in Three Runtimes

Documented support, then observed resolution

Read agent, model, tools, and checks from each runtime's artifacts

PART 3

Prove and Place

Load, compare, place, rerun

Map each gap to its owning layer and owner, then rerun the proof

PART 4

Take It to Your Agents

Own-work check and next moves

Run the load report on an agent your team already shares

One Agent File Entered Three Runtimes and Produced Three Stories

FanHub, 2026-10-21: the same .github/agents/test-writer.agent.md, tests for one small component each

VS Code	Agent declared Claude Sonnet 5.5; its requests ran gpt-6.1-sol	`#216 · request log`
Copilot CLI	Logged: Using model claude-sonnet-5.5 from custom agent test-writer	`#215 · debug log`
Cloud agent	Ran test-writer; Jest stopped at Cannot find module 'react'	`#214 · session log`

All three runtimes recognized and invoked the agent. Which agent did the team ship?

PART 1

Design the Portable Core

Decide when a role earns a file, split description, tools, and body, then publish where it loads



A different who

An agent file carries a role: identity, tool boundary, method, model



The body travels

Every runtime reads the body, so the output contract lives there



Most local wins

Identify the winning file before debugging the prompt

```
.github/agents/test-writer.agent.md · committed to main (df50ea1)
↳ description selects · tools bound · body teaches
```

A Custom Agent Earns Its File When the Work Needs a Different Who

📜

Instructions

Conventions every agent follows: copilot-instructions.md, *.instructions.md, AGENTS.md

🎯

Prompt file

One repeatable task a person starts by name (*.prompt.md)

🧠

Skill

A method plus scripts and resources any agent loads on demand (SKILL.md)

Custom agent

A persistent role: identity, tool boundary, method, and model preference (*.agent.md)

🧪 test-writer edits test files only, runs one command family, and reports production bugs for a person to fix.

Description Selects, Tools Bound, Body Teaches

.github/agents/test-writer.agent.md (excerpt)

```
---
name: test-writer
description: Writes focused Jest tests for one FanHub Node
  frontend component or page, runs them, and reports any
  production behavior that looks wrong.
  Edits test files only.
tools: ["read", "search", "edit", "execute", "todo", "web"]
model: "Claude Sonnet 5.5 (copilot)"
handoffs:
  - label: Review these tests
    agent: test-reviewer
    send: false
---
# Test writer for the FanHub Node frontend
You write tests. You do not change production code.
## Scope … ## Method … ## Final summary …
```

🎯 description selects

The only required field. Pickers show it; the CLI and VS Code use it to choose an agent

🧰 tools bound

Aliases each runtime maps to its own tools. tools: [] grants none; omitting it grants all

📘 body teaches

Method and output contract, up to 30,000 characters. Every runtime reads it

Write the Output Contract Into the Body

test-writer final summary · five lines, every run

```
Agent:              test-writer
Tests added:        <test file and behaviors>
Command and result: <command> → <runner counts>
Suspected bugs:     <production behavior for a person>
Files changed:      <test files only>
```

Cloud: the summary carried the failure

"could not run tests because react was missing"

The PR description told another part

It covered the behaviors tested; the agent summary held the execution result

Portable by construction

Every runtime reads the body, so every teammate gets the same five lines

Identify the Winning File Before Debugging the Prompt

The filename is the agent ID; when two scopes define one ID, the most local definition wins

Repository	.github/agents/*.agent.md: everyone in the repo, every runtime	`team default`
Personal	~/.copilot/agents/ for the CLI; VS Code profile storage for VS Code	`one developer`
Organization	agents/ in the org .github or .github-private repo; cloud and VS Code	`every org repo`
Precedence	Personal, then repository, then organization, then enterprise	`most local wins`
Invocation	target, user-invocable: false, disable-model-invocation: true	`who can start it`

The cloud agent reads agents from the default branch: FanHub committed test-writer to main (df50ea1) first

PART 2

Run One File in Three Runtimes

Separate documented support from observed resolution, then read what each runtime did with test-writer



Documented support

Runtimes skip unsupported fields quietly, by design



Observed resolution

FanHub #216, #215, and #214 on 2026-10-21



Environment dependency

Setup steps outside the profile let cloud Jest run

```
gh agent-task create --custom-agent test-writer → #214
↳ Cannot find module 'react' → setup steps → 6 passed
```

EVIDENCE: DOCUMENTED SUPPORT

Runtimes Skip Unsupported Fields Quietly, by Design

Documented support on 2026-10-21, from the custom agents configuration reference

All three	name, description, tools, target, and both invocation controls	VS Code · CLI · cloud
model	VS Code and the CLI read it; cloud docs list it for IDE use	VS Code · CLI
VS Code	handoffs, agents, argument-hint (Local); reasoning-effort	VS Code only
Agent-scoped hooks	hooks in the agent file run in VS Code and the CLI	VS Code · CLI
Copilot CLI	include-custom-instructions for subagent runs	CLI only
Cloud agent	mcp-servers, metadata	cloud only

Quiet skipping lets one profile serve several products without errors

Observed Resolution: One File, Resolved Three Ways

VS Code · #216

- model: requests ran gpt-6.1-sol; cause unresolved
- tools: 6 aliases → 23 VS Code tools
- handoffs + argument-hint: both worked in a Local session
- Agent test run · runner: 14 passed

CLI 1.0.100 · #215

- model: resolved to claude-sonnet-5.5, logged
- tools: 6 aliases → 14 CLI tools
- handoffs + argument-hint: skipped; nothing logged
- Agent test run · runner: 10 passed

Cloud agent · #214

- model: unobserved; the session log names no model
- tools: ran as a subagent; count unobserved
- handoffs + argument-hint: skipped; nothing logged
- Agent test run · stopped at missing react; after setup steps, 6 passed

Tool counts are each runtime's own expansion of the same six aliases. Unobserved marks what the artifacts leave open.

VS Code Honored Most Fields; Requests Ran Another Model

✅ Observed resolution · Local session

argument-hint + handoff
Hint shown in a Local session; Review these tests opened test-reviewer

23 tools · 10 tools
test-writer from six aliases; test-reviewer from read and search, read-only

Agent results
Runner: 14 passed · reviewer verdict: accept

❓ Unresolved discrepancy · model

Declared
model: Claude Sonnet 5.5 (copilot)

Executed
gpt-6.1-sol on every request, per the request log

Cause unresolved
The evidence does not identify why; only the request log revealed it

🔎 Read the model from the run whenever the model matters.

The Copilot CLI Resolved the Model and Logged Its Decision

```
copilot --agent test-writer --log-level debug
```

```
Using model "claude-sonnet-5.5"
  from custom agent "test-writer"
  (resolved from "Claude Sonnet 5.5 (copilot)")
```

🧠 Model: resolved and logged

Declared Claude Sonnet 5.5 (copilot) became claude-sonnet-5.5, in writing

🧰 6 aliases → 14 CLI tools

The CLI's own expansion, including web_fetch and update_todo

⏭️ handoffs, argument-hint skipped

The log has no entry for either field

🧪 Agent test run

Runner: 10 passed · test file only · 8.65 AI Credits in 2 min 14 s

Cloud Agent Ran test-writer; Jest Stopped at Missing react

Cloud run · execution-environment dependency

Agent
PR: Custom agent used: test-writer · log: Begin subagent: test-writer

Runner
Six tests written; Jest stopped at Cannot find module 'react'

Cause
Fresh clone, no setup steps; the agent's npm ci could not reach its package proxy

Same tests, run locally

Runner
5 passed, 1 failed

Domain result
One card-wide no-Joey assertion failed; the quote's context line names Joey

Reports
Agent summary carried the failure; PR description covered behaviors tested

The agent created tests and the tests are valid are two claims, each with its own proof.

Setup Steps Outside the Profile Let Cloud Jest Pass 6 of 6

❌ First cloud run stopped

The cloud runner starts from a fresh clone

No frontend dependencies
FanHub had no setup steps for the Node frontend

One broad assertion
The Joey test failed locally

💡 FanHub prepared the runtime

copilot-setup-steps.yml
Installs frontend dependencies · e1675df · validation run passed

@copilot comment on #214
Fix the Joey test and report exact counts

✅ Cloud rerun, observed

test-writer started again; Jest ran in the cloud

Code review subagent reran the file: same result

Fix bf7e2c3 asserts on .attribution; local rerun 6 of 6

Counts live in the reply comment; PR description kept its first text

1 suite · 6 tests
passed in the cloud rerun

💡 An agent profile describes a role; the runtime supplies the environment.

PART 3

Prove the Run and Place Each Requirement

Fill a load report per runtime, place each gap in its owning layer, and rerun the same proof

📋

Load report

Five questions by three runtimes, each with its debug source

🔁

Load, compare, place, rerun

Each gap gets an owning layer and a named owner

🤝

Producer + checker

test-writer hands off to read-only test-reviewer

```
gh agent-task view <session> --log
↳ which agent · which model · which tools · which checks · what it reported
```

Fill a Load Report From Every Runtime's Own Artifacts

VS Code

Chat Debug view · Agent Debug Log (preview)

- **Agent: picker; Load Agents event in the Agent Debug Log**
- **Model: Chat Debug view or prompt export, per request**
- **Tools: tool list sent with each request**
- **Checks: terminal output in chat**
- **Report: final summary**

Copilot CLI

--log-level debug --log-dir <dir> · --share

- **Agent: /agent list; agent_name in the debug log**
- **Model: Using model … from custom agent line**
- **Tools: tool_names in the debug log**
- **Checks: session output**
- **Report: final summary**

Cloud agent

gh agent-task view <session> --log

- **Agent: Custom agent used, in the PR description**
- **Model: session log, when shown**
- **Tools: session log tool calls**
- **Checks: session log commands and exit codes**
- **Report: agent summary vs the PR description**

Five questions per runtime: which agent, model, and tools ran; did its checks run; what did it report.

Load, Compare, Place, Rerun: Each Gap Has a Layer and an Owner

Read the resolved run, compare it with the profile, place the gap, and rerun the same proof

Skipped field	Restate handoffs behavior in the body so every runtime carries it	`agent body · producer`
Model differs	Treat model as a preference; pin it at launch (picker, --model)	`launch · developer`
Alias mapped	Build on tools every runtime provides, or scope with target	`body or target · producer`
Missing deps	Prepare outside the profile: setup steps, dev container, script	`environment · maintainer`
Missing proof	Keep the output contract; accept work on command evidence	`contract + review · reviewer`
Variance	Agree on supported runtimes and which differences escalate	`team policy · team lead`

Load → compare → place → rerun: one targeted change in the owning layer, then the same proof again

Pair a Narrow Producer With a Read-Only Checker

🤝 Handoffs · VS Code Local

A person between steps
A button with a prefilled prompt, sent when the person chooses

test-writer → test-reviewer
The producer edits test files; the checker reads and searches only

🧩 Subagents · an agent calls an agent

VS Code
The agents field plus the agent tool limits which agents a coordinator calls

Copilot CLI
Delegates by description; subagents get repo instructions with include-custom-instructions: true

Cloud agent
Ran test-writer itself as a subagent

🔐 The agent file shapes what the agent attempts. Rulesets, branch protection, hooks, permission flags, and the firewall hold authority over changes.

Run the Load Report on an Agent Your Team Already Shares

🎯 Your own-work check

Pick
One agent your team already shares

Predict
Which load-report line differs first in another runtime

Run
Each runtime the team uses, with the debug sources on

Place
The gap in its owning layer, then rerun the same proof

🧭 Where it stops fitting

Local Agentic SDLC
An orchestrator calling several agents in worktrees

GitHub Agentic Workflows
Event-triggered judgment with safe outputs

Copilot on GitHub
The PR decision across cloud agent, code review, and Code Quality

❓ Which of your agents would behave differently tomorrow from another runtime, and which load-report line would show it first?

From One File Assumed Everywhere to a Resolved Run Read Everywhere

Before

- One agent file assumed to behave the same in every runtime
- Declared model read as the model that ran
- Key behavior placed in runtime-specific fields
- Agent summary or PR text accepted as the result

After

- A portable core in the body; runtime fields added on purpose
- The model read from each runtime's run artifacts
- Each gap placed in its owning layer with a named owner
- Command evidence alongside generated prose, then a rerun

23 · 14

Tools from the same six aliases: VS Code and CLI expansions

Cause unresolved

VS Code ran gpt-6.1-sol; agent declared Claude Sonnet 5.5

6 passed

Cloud Jest rerun after copilot-setup-steps.yml

✅ WHAT YOU CAN DO TODAY

Today

- Check one shared agent's frontmatter against the documented support matrix
- Find the winning file for that agent ID across personal, repository, and org scopes
- Open one debug source: Chat Debug view, --log-level debug, or gh agent-task view --log

This Week

- Run the agent in each runtime the team uses and fill in the load report
- Move one runtime-only behavior into the body, then rerun the same proof
- Add or check copilot-setup-steps.yml so cloud runs can execute the agent's checks

This Month

- Name an owner for each layer: body, launch, environment, review, policy
- Agree which runtimes each shared agent supports and which variance escalates
- Pair one producer agent with a read-only checker

🔑 Key Takeaway

Design the portable core, read the resolved run, and place each requirement in the layer that owns it.

REFERENCES

Custom Agent Reference

- Custom agents configuration reference
  Fields, aliases, quiet skipping
- About custom agents
  Concepts and scopes
- Comparing customizations
  When a role earns a file
- VS Code: custom agent file reference
  VS Code fields

Runtimes and Run Evidence

- Create custom agents for Copilot CLI
  CLI setup
- Create custom agents for cloud agent
  Cloud setup
- Customize the cloud agent environment
  copilot-setup-steps.yml
- VS Code: custom agents
  Handoffs and subagents
- VS Code: Chat Debug view
  Agent Debug Log

FanHub Evidence

- test-writer.agent.md
  The agent file
- #216

Related Talks

- Copilot Primitives
  Agents among instructions, prompts, skills, hooks
- Copilot on GitHub



Custom Agents

ONE FILE, EVERY RUNTIME. DESIGN THE CORE, READ THE RUN, PLACE EACH REQUIREMENT

6 aliases → 23 · 14

Each runtime expanded the same tools field its own way

claude-sonnet-5.5

The Copilot CLI logged its model decision from test-writer

6 passed

Cloud Jest rerun after a setup workflow outside the profile

Which of your shared agents would behave differently tomorrow from another runtime, and which load-report line would show it first?